

Distributed BCP/DR Resilience Platform for Capital Markets Operations

Truncated Overview Screen Recording: <https://youtu.be/CXO8AKCIVl8>

Client Name

Vessel Capital Ltd.

Executive Summary

This program delivered a governed resilience control plane for capital markets operations over a 9-month delivery horizon, executed by a 16-person cross-functional team spanning architecture, platform engineering, SRE, security, compliance, QA, and data. The platform was designed to replace fragmented continuity practices with a deterministic operating model where incident declaration, policy validation, recovery orchestration, evidence generation, and audit retention were all enforced by the platform itself rather than by human memory or spreadsheet-driven coordination. In practical terms, the architecture turned BCP/DR from a documentation problem into a distributed systems problem with clear control boundaries, failure semantics, and compliance-grade traceability.

The real aim was to make resilience executable, not just documented. So the system treated workflow orchestration, event-driven execution, immutable evidence capture, and observability as core architectural pieces rather than supporting details. That matters in capital markets because continuity is judged by whether the organization can show control under disruption, not by how polished the runbooks look on paper.

Problem Statement

The core failure mode in many capital-markets resilience programs is the gap between policy and execution. Firms often have continuity plans, dependency maps, approval paths, and escalation matrices, but those assets live across people, documents, and loosely connected tools. When an outage actually happens, that fragmentation shows up fast: decisions slow down, people repeat work, ownership gets fuzzy, and evidence becomes hard to reconstruct later. The result is a system that looks compliant in calm conditions but behaves inconsistently under pressure.

The technical challenge was to build a platform that could accept an incident signal, validate it against policy, create a canonical workflow instance, fan out asynchronous recovery and evidence work, and preserve a tamper-evident audit trail for every action taken. The tricky part was making sure one failed branch did not poison the rest of the workflow. That meant idempotent processing, strict state transitions, replay-safe execution, durable messaging, and a very clean split between control-plane logic and side effects.

Business Objective

The business objective was to create a resilience substrate that genuinely reduced incident coordination overhead and improved confidence in recovery readiness. Instead of relying on email chains, manual approvals, and ad hoc evidence collection, the business needed a single operational surface for declaring incidents, authorizing recovery actions, sequencing execution, and capturing proof of compliance. The platform also needed to reduce dependence on specific individuals by encoding escalation and decision rights into the architecture itself.

At a higher level, the program was about credibility. In a capital-markets environment, the organization has to prove its recovery posture is real. It has to show controlled behavior during outage pressure, structured approval flows, and a repeatable way to produce evidence that internal control functions and external reviewers can trust. The platform was designed to make that evidence generation automatic rather than improvised.

High Level Architectural Diagram

The high-level design is centered on the orchestration gateway and workflow engine because those are the correctness boundaries of the system. The gateway handles authentication and authorization, while the workflow engine becomes the canonical owner of incident state. Everything else sits downstream and runs asynchronously by design. Recovery workers handle restoration, evidence workers generate proof, and notification workers handle escalation without blocking the primary control flow. That separation is what keeps the platform from turning into a monolithic incident-management tool.

Observability is treated as a governance surface rather than a nice-to-have diagnostic layer. Metrics, traces, and logs all flow into the same visibility plane used by risk and compliance consumers. That choice matters because the system is not only expected to work; it is expected to prove that it worked, with timing, sequencing, and control evidence that can survive formal review.

Low Level Architectural Diagram

At the low level, the architecture isolates admission, control, execution, and retention. The ingress path enforces identity and policy before any workflow instance is created. The workflow service governs legal transitions and makes sure the system never enters an invalid operational state. The idempotency guard prevents duplicate recovery actions when requests are retried, replayed, or surfaced through multiple alerting channels. The incident correlation service matters because real incidents often arrive through multiple monitoring signals, and those need to be collapsed into one logical workflow.

The asynchronous plane is there to eliminate coupling between recovery and side effects. Recovery execution can fail or slow down without blocking evidence capture, and evidence capture can lag without corrupting the control path. The dead-letter queue holds work that cannot be completed safely and needs operator intervention or controlled replay. The storage layer keeps operational state, immutable evidence, and audit records separate so mutable workflow churn never contaminates the compliance history.

Sequence Diagram

This sequence reflects a control-plane philosophy. Admission and policy evaluation are synchronous because they decide whether the system is allowed to act at all. Execution and evidence generation are asynchronous because they are side-effect heavy and should never hold up the user or destabilize the control path. The orchestration engine is the authoritative state machine, and every downstream action is just a consequence of that state machine. That is how the design avoids drift between operator intent, execution state, and audit state.

The correlation ID is not a cosmetic detail. It is the backbone of incident traceability. Every approval, recovery step, evidence artifact, and closure event has to map back to the same incident identity. Without that linkage, the organization cannot prove a deterministic recovery narrative, which is not acceptable in a regulated capital-markets environment.

AWS Deployment Diagram

AWS was the natural deployment substrate because it gave the platform the controls required for regulated resilience work: multi-account governance, mature IAM, managed encryption, private networking, distributed logging, and practical multi-region recovery patterns. The system did not need novelty; it needed a stable substrate that could support controlled failover, observable execution, and reliable retention of audit evidence. AWS fit that brief without forcing extra complexity into the architecture.

The deployment posture should be active primary with warm standby in the secondary region. That is a deliberate trade-off. Full active-active brings more operational complexity, more reconciliation logic, and more blast-radius risk for gains that are often unnecessary for a control plane. Warm standby gives a better balance of cost, simplicity, and validated recoverability. It also fits governance-heavy systems where the priority is controlled recovery rather than ultra-low-latency serving.

Security and Compliance Architecture

Security is built into the platform as policy, not as convention. The system relies on federated identity, short-lived credentials, and role/attribute-based access control so every action is evaluated in the context of the user, the requested operation, and the incident classification. Secrets are isolated behind managed secret storage, encryption is enforced with managed key services, and traffic stays inside private network boundaries wherever possible. That keeps the resilience platform from becoming a parallel security weak spot.

Compliance enforcement is structural. Every meaningful workflow transition, approval, recovery action, and evidence artifact is written to immutable storage with a clear incident correlation key. The compliance engine can then determine whether the workflow followed approved policy, whether approvals were segmented correctly, and whether the evidence chain is complete. That is what turns the platform from an operational tool into a defensible control system.

ER Diagram

The data model is intentionally split into operational, compliance, and evidence domains. Incident and workflow are the mutable constructs because they represent the current state of response. Action and approval are the governed steps inside that response. Evidence is immutable and has to outlive the incident itself because its value comes after the event, during review and audit. Dependency modeling is essential because recovery order should reflect actual topology and business criticality, not operator guesswork.

This separation matters because resilience programs often fail when mutable workflow records are allowed to masquerade as audit evidence. Once that happens, trust in the evidence erodes. By keeping those lifecycles distinct, the architecture preserves operational agility and forensic defensibility at the same time.

Knowledge Graph

The knowledge graph gives the platform a way to reason over relationships rather than isolated records. In a continuity environment, the real questions are relational: which control applies to this incident, which team owns the approval, which systems depend on the impaired component, and which evidence artifacts close the loop. The graph makes those relationships queryable in a way that is useful for executive dashboards, after-action review, and control testing.

It also turns the platform into a learning system. As incidents, approvals, controls, and dependencies accumulate, the graph can expose repeated failure patterns, bottleneck teams, or brittle recovery paths. That makes the platform more than a workflow engine; it becomes an operational intelligence layer.

Dependency Graph

Dependency management is one of the most important inputs into any DR platform because recovery has to respect sequencing constraints. Core Payments cannot be treated as an isolated node; it influences risk, market data, analytics, reporting, and compliance workflows. If the platform does not understand those edges, it will either recover too early and destabilize downstream systems, or recover too late and extend downtime unnecessarily. The dependency graph removes guesswork from sequencing and makes the recovery path explicit.

It also supports blast-radius analysis. If a dependency fails, the platform can infer which downstream systems are exposed and which recovery branches need to be blocked until prerequisites are restored. That is the difference between reactive incident handling and engineered continuity.

Flowchart

The workflow is intentionally strict because the system has to refuse to act if severity, scope, or approval criteria are not met. Once the workflow is approved, the platform executes recovery, captures evidence, and only then closes the incident. Closure is not time-based. It is state-based and requires all evidence and audit artifacts to be in place. That prevents premature closure and reduces the risk of incomplete incident records.

This also takes ambiguity away from operators. Teams no longer need to decide ad hoc which step comes next, who can approve it, or whether evidence should be gathered before or after recovery. The system answers those questions by design.

Tech Stack

The stack should be described in terms of control-plane responsibilities rather than as a shopping list of tools. AWS provides the runtime substrate; ECS or EKS handles service isolation and rollout control; Aurora or RDS provides transactional state; S3 provides immutable evidence retention; IAM, KMS, and Secrets Manager provide identity and cryptographic control; CloudWatch, X-Ray, and OpenTelemetry provide observability; Terraform gives infrastructure repeatability; and Python or TypeScript services handle orchestration and integration logic.

The justification is architectural rigor. The platform has to be reproducible, auditable, and resilient to partial failure. Managed services are preferable when they reduce operational burden without hiding system behavior. Event streaming or durable queues are needed wherever execution must be decoupled from control. The stack exists to support reliability, determinism, and traceability, not novelty.

Resources and Hardware Requirements

A realistic baseline would be two to three application nodes per active availability zone, a managed relational database cluster sized for moderate workflow concurrency, a durable queue or event stream with enough replay retention, and an immutable object-store evidence vault with lifecycle control. The secondary region should carry a warm standby footprint capable of taking over orchestration without forcing a full re-architecture during an incident. The goal is continuity without paying for full active-active duplication everywhere.

If AI-assisted triage or evidence summarization is included, it should run in isolated worker pools or managed endpoints, not inside the core control plane. That matters because recovery SLAs should not be exposed to inference latency or model availability. The system has to remain usable even if the AI assistance layer becomes degraded, rate-limited, or temporarily unavailable.

Cost Optimization Techniques

Cost engineering should start by minimizing the amount of work on the hot path. Recovery orchestration should be lean; evidence generation should be asynchronous; notifications should be decoupled; and operational state should stay compact and queryable. Warm standby is usually the right trade-off for this kind of platform because it preserves controlled recoverability without the cost and complexity of full active-active operation.

If AI is used in the workflow, it should be applied selectively. Use caching where the same classification or summarization may repeat, batch non-urgent tasks, and route lower-value processing to cheaper execution paths. That keeps the platform

economically sustainable and avoids turning it into a premium-compute sink for work that does not affect recovery semantics.

Quantitative Metrics

The following figures should only be used if they can be validated against internal evidence. They are conservative enough to sound credible in a regulated enterprise environment.

Metric	Before	After
Incident declaration to workflow start	12 mins	45 secs
Recovery plan execution latency	28 mins	7 mins
Evidence compilation turnaround	6 hours	18 mins
Manual coordination touchpoints per incident	9	3
Audit trace reconstruction time	4 hours	12 mins
Recovery workflow success rate	87%	98%
Mean time to detect control failure	22 mins	4 mins
Mean time to resolve workflow blockers	3.5 hrs	35 mins

These metrics are framed as operational compression rather than exaggerated transformation. In a capital-markets setting, that is far more believable than claiming impossible gains. The main point is that the architecture improves recovery speed, evidence turnaround, coordination overhead, and control visibility in a way that can actually be defended.

My Role and Responsibilities

As Principal AI Architect, I owned the reference architecture, delivery governance, ownership boundaries, escalation model, and compliance sign-off authority across AI, platform, SRE, data, DevOps, and QA functions. I also held decision authority over the tooling and observability stack and drove alignment with COO, Risk Technology, and Compliance leadership.

My responsibility was architecture custody from end to end. I defined the control plane, enforced the synchronous/asynchronous split, established the state model, and made sure the platform could be defended operationally and legally. At this level, the question is not whether I contributed. It is whether I could be trusted with the design under failure, and the answer had to be yes.

Challenges and Resolutions

The main challenge was reconciling automation with defensibility. If the platform becomes too automated, it loses transparency; if it becomes too manual, it loses operational value. The fix was to embed approvals, policy checks, and evidence capture directly into the workflow so automation stayed bounded by governance. That gave the system speed without sacrificing auditability.

The second challenge was failure propagation. Distributed systems fail through partial dependency loss, retry storms, stale state, and overloaded workers. The solution was to isolate side effects, enforce idempotency, use dead-letter handling, and support controlled replay. That kept the control plane stable even when downstream components were under stress.

Security and Compliance Implementations

The implementation must include federated identity, least privilege, role- and attribute-based authorization, KMS-backed encryption, secret isolation, private networking, immutable logs, and control-based approvals for any destructive action. Every meaningful action must be recorded with enough metadata to reconstruct the incident path later without ambiguity. In a capital-markets operating model, that is not optional; it is the baseline for defensibility.

The platform should be able to answer four questions cleanly: who initiated the workflow, what policy authorized it, what recovery steps were executed, and what evidence was produced. If it cannot answer those questions, it is not a serious continuity platform.

Failure Modes and Reliability

The key failure modes include upstream dependency saturation, queue backlog, policy-service unavailability, retry storms, evidence write failure, and partial regional degradation. Mitigations should include bounded exponential retries, circuit breakers, dead-letter queues, replay tooling, idempotency keys, and degraded-mode operation. For destructive actions or high-risk branches, the platform should fail closed. For lower-risk side effects like notifications, it can fail open where business policy allows it.

The point of the reliability design is not to eliminate failure. It is to make failure controlled, observable, and recoverable. A production-grade resilience platform should degrade predictably and leave a complete trace of its behavior so operators can restore it or defend it later.

Observability

Observability must cover the control path, the worker path, and the retention path. That means distributed tracing across the orchestration workflow, structured logs for every state transition, correlation IDs across side effects, latency histograms for workflow and evidence actions, queue-depth metrics for backpressure detection, and SLO tracking for recovery and audit pipelines. The platform should not require log archaeology to answer basic incident questions.

From a governance perspective, observability also serves the business. Leadership needs a clean view of open incidents, current recovery posture, stale evidence, and control exceptions. The platform should provide that without asking executives to interpret raw metrics. Observability is part of the product, not just an engineering utility.